superjoin

HR onboarding agent

Architecture, microservices, API reference and project details

Repository: github.com/akshat1423/superHR · branch worktree-restyle-paper-theme

Document generated 31 October 2026 from the source tree

Next.js 16 · React 19 · Supabase · Anthropic · Vercel

Contents

1. Overview 3
2. System architecture 4
3. How the pieces fit 5
4. Microservice catalogue 6
5. Provisioning microservices 8
6. Provisioning run and logging 10
7. Documents, changes and termination 11
8. ATS sync, chat agent, assistant, escalation 12
9. API reference: auth, health, logs, email 13
10. API reference: assistant, documents, ATS 15
11. API reference: services, chat, pages 17
12. Data model and storage 18
13. Security and safety 19
14. Domain logic in detail 20
15. Roles, permissions and frontend 21
16. Configuration and deployment 22
17. Testing and repository layout 23
18. What is simulated, limits and next steps 24

> The service tables in this document are parsed from the source code when it is generated, so they match the code. Re-generate with `python3 docs/build_architecture_pdf.py`.

Overview

superjoin is an HR onboarding agent. It brings new joiners in from an applicant-tracking system (ATS), a form or a file; builds a rule-based checklist for each person that says *why* every item is there; collects and validates their documents with a document service; provisions their accounts, access, payroll, desk and day-one plan through ten provisioning microservices; chases overdue work and escalates it to the right owner; and lets HR undo, change or terminate any of it.

What it does

- **Ingest**: sync hired candidates from Greenhouse, Lever or Workday (page by page, with a request log), add one joiner with a form, or upload CSV, Excel or JSON. Every route in goes through the same validation (mandatory fields, dates, manager and department references).
- **Checklists**: requirements are data (conditions on department, location, country, role, grade, type, with dependencies). Each item states the rule behind it and its owner. States: completed, pending, blocked, overdue.
- **Documents**: uploads go to a document service that reads the fields, checks them against the profile with our own code, and decides: Accepted, or Correction Requested with reasons. Under Review is left for a scan nobody can read.
- **Provisioning**: ten services (email and SSO, offer letter, app access, background check, payroll, ID card, desk, day-one agenda, document collection, reminders), each a server function with a sandbox or a live (webhook) connector.
- **Change and undo**: deprovision or re-provision any service for one person, switch single apps on or off, terminate and reinstate an onboarding, and preview and apply what a change of date, role, department or location means for provisioning.
- **Assistants**: a policy-grounded assistant per person (cites the policy or says it isn't covered) and a role-scoped chat agent that can look things up and, with a confirming click, retry a service or send an escalation.
- **Dashboards**: a filterable progress table, an ID-badge view, a workflow builder and library (with role-scoped departments), reminders by email, and agent logs with per-service drill-down and failure reasons.

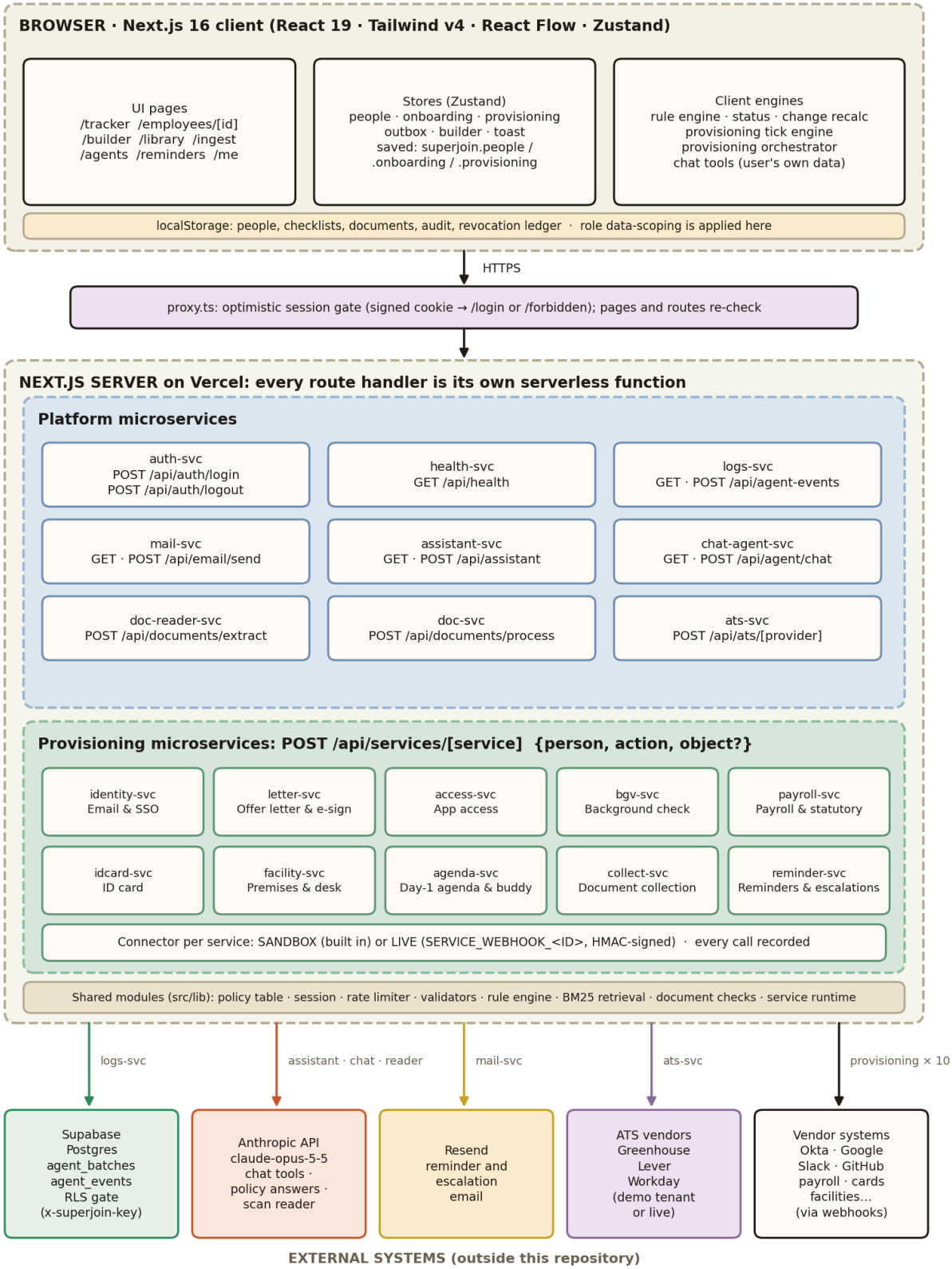
Technology

Layer	Choice
Framework	Next.js 16 (App Router, Cache Components, proxy.ts), React 19, TypeScript, Node 20.9 or later
UI	Tailwind CSS v4, React Flow (workflow builder), brand logos generated from icon sets
Client state	Zustand with persist (people, onboarding data, provisioning ledger), localStorage
Auth	Signed HttpOnly session cookie (jose, HS256, 8 h, user id only), five demo personas, policy table
Backend	Next.js route handlers (serverless functions on Vercel), no separate server
Database	Supabase Postgres for agent logs (append-only, row-level security behind a server-held key)
AI	Anthropic API (claude-opus-5-5 by default) for the chat agent, policy answers and scan reading; everything also works without a key
Email	Resend (or preview mode)
Quality	481 unit checks, 120 end-to-end checks, ESLint, TypeScript strict, production build

Demo user	Role	Opens	Sees
Yash Bansal	New joiner (employee)	My onboarding	only their own record
Arjun Rao	Manager	Tracker	direct reports; workflows of their reports' departments, read-only
Vikram Singh	Head of Engineering	Tracker	Engineering people and workflows; edits Engineering workflows
Neha Kapoor	Head of Finance	Tracker	Finance people and workflows; edits Finance workflows
Priya Nair	HR	Tracker	everyone; the only one who can ingest, publish workflows or terminate an onboarding

System architecture

The browser holds the onboarding data and runs the rule and provisioning engines; the server is a set of small, independently deployed functions that do the work that needs a secret, a vendor or a model. The diagram shows the layers, the microservices, and every external system.



How the pieces fit

Layers and trust boundaries

Layer	Responsibility	Trusted for
Browser	UI, Zustand stores, rule engine, status derivation, provisioning tick engine, chat tool execution, localStorage persistence	nothing: all input is re-validated on the server
proxy.ts	Optimistic gate: signed-out visitors go to /login; a role that can't open an area is rewritten to /forbidden. API routes and static samples are covered by the same matcher rules	UX only, not a security boundary
Route handlers	Verify the session, check role and department scope, rate-limit, validate and length-limit input, then act	authentication, authorisation, validation
Supabase	Stores agent events; row-level security requires a header (x-superjoin-key) that only our server knows	append-only log storage
Anthropic	Reads scans (fields only) and writes answers from supplied sources; never makes a decision	nothing: output is verified or only read
Vendors	Real systems behind the provisioning services and the ATS; reached only through server-side connectors	their own side effects

“Pods” as microservices: what that means here

Each microservice is a **route handler deployed as its own serverless function** on Vercel. It scales, times out (60 s on the AI and service routes) and fails independently, owns one responsibility, and communicates with the others only over HTTP or through the browser. They are logical microservices in one repository, sharing code from `src/lib`; they are not Kubernetes pods. Because every service already has a narrow HTTP contract (and the provisioning ones can be pointed at an external system with a webhook), any of them can be moved into its own container or cluster without changing its callers.

> **Rate limits are in memory**, so they apply per function instance. Put the limiter in Redis or Upstash before running many instances.

Where state lives

State	Where	Survives reload?	Shared across users?
People, checklists, documents, audit trail, reference data (rules, policies, owners)	browser localStorage (superjoin.people, superjoin.onboarding)	yes	no (per browser)
Provisioning run (tasks, calls, events)	browser memory (rebuilt on load)	no	no
Revocations and terminations	browser localStorage (superjoin.provisioning ledger), re-applied on load	yes	no
Agent events (logs)	Supabase agent_batches, agent_events	yes	yes (HR sees all; department heads their department)
Session	signed cookie sj_session	8 hours	n/a

Microservice catalogue

Nineteen microservices: nine platform services and ten provisioning services. Each row is one deployable unit.

Platform microservices

Service	Endpoint(s)	Responsibility	Calls out to	Timeout
auth-svc	`POST /api/auth/login` `POST /api/auth/logout`	Demo sign-in by persona; sets or clears the signed session cookie. Swap for an SSO callback in production	none	default
health-svc	`GET /api/health`	Public liveness; for HR also which integrations are configured and each service's connector mode (booleans only, never values)	none	default
logs-svc	`GET /api/agent-events` `POST /api/agent-events`	Reads and appends agent events, scoped by role; idempotent writes	Supabase	default
mail-svc	`GET /api/email/send` `POST /api/email/send`	Sends reminder and escalation email; preview mode without a key; can funnel all mail to one inbox for testing	Resend	default
assistant-svc	`GET /api/assistant` `POST /api/assistant`	Answers a person's question from approved policies and their own checklist; cites sources or says not covered	Anthropic (optional)	60 s
chat-agent-svc	`GET /api/agent/chat` `POST /api/agent/chat`	One model step of the role-scoped chat agent (tools run in the browser)	Anthropic	60 s
doc-reader-svc	`POST /api/documents/extract`	Reads the printed fields off a PDF or image; returns fields only	Anthropic	60 s
doc-svc	`POST /api/documents/process`	Document service: receive, read, check against the profile, decide	doc-reader logic, Anthropic (scans)	60 s
ats-svc	`POST /api/ats/[provider]`	Fetches one page of hired candidates from Greenhouse, Lever or Workday (demo tenant or live)	ATS vendor	60 s

Provisioning microservices

All ten share one endpoint shape, `POST /api/services/[service]`, and one runtime. Each defines its own calls, reverse calls, update calls and failure modes. Details of each follow in the next section.

Service	Creates	Depends on	Calls	Failure modes
`identity-svc`	Google Workspace mailbox, Okta user	none	3	409 Mailbox alias already exists; 429 Directory API rate limit hit
`letter-svc`	renders the offer PDF and sends the e-sign link	none	2	500 Offer template variable missing
`access-svc`	Slack, GitHub, Jira, Claude… at the persona's permission level	identity-svc	3	403 GitHub organisation is out of seats; 404 Okta group not found
`bgv-svc`	education / employment / address checks	letter-svc	2	504 Verification vendor timed out
`payroll-svc`	payroll record, PF/UAN, insurance enrolment	none	3	422 Bank IFSC code failed validation; 409 Duplicate employee code
`idcard-svc`	queues the print job with photo and QR	none	2	422 Photo too small for printing
`facility-svc`	building access, desk, parking	idcard-svc	2	409 No free desks on the floor
`agenda-svc`	builds the agenda, assigns a buddy, drafts the intro post	identity-svc	2	502 Manager calendar unavailable
`collect-svc`	magic link for missing documents and fields	none	2	400 WhatsApp number not opted in

`reminder-svc`	schedules every T+N nudge in the workflow	all	2	550 Recipient mailbox rejected the test email

Internal modules (not network services)

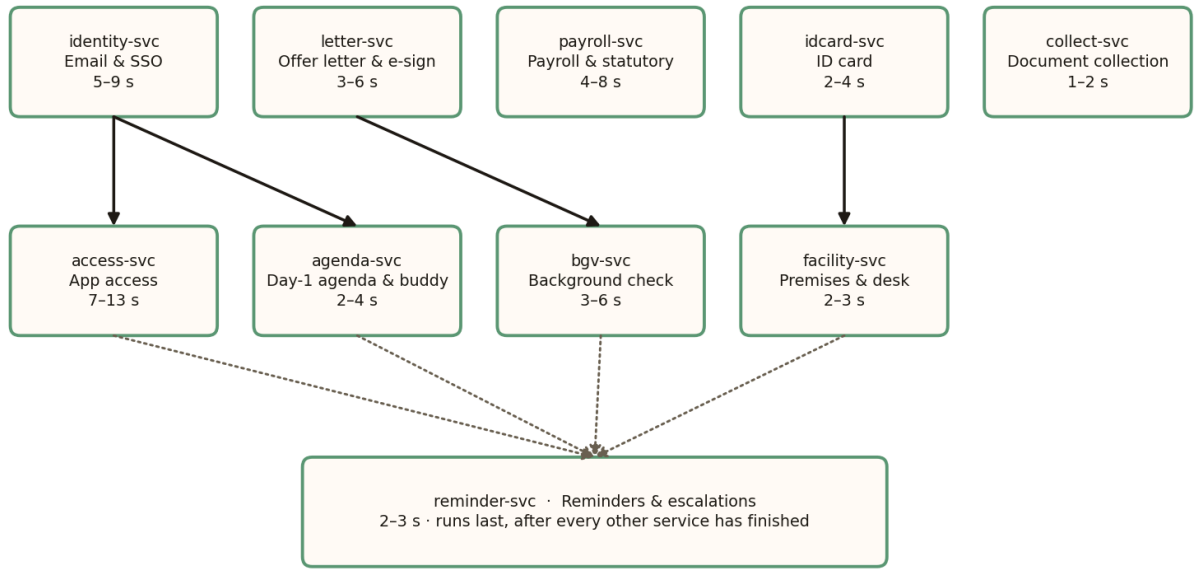
Module	Role
rules / status / change / escalate	Checklist generation with reasons, derived states, recalculation after a change, escalation targets
datasets / csv / documents / docService	Parsing and validating profiles, requirements, policies and owners; document checks; the document decision
retrieve / assistant	BM25 retrieval with a coverage gate, grounded answers, citation verification
agent (tools, offline, prompt)	Role-scoped chat tools, the offline router, the system prompt
serviceRuntime (runner, handlers, reverse)	Executes a service action in sandbox or live mode and records every call
provisioning (impact, orchestrate)	What a change means for provisioning; running actions, terminate and reinstate
auth (policy, token, session, users)	The single policy table, session signing, the data-scoping rule

Provisioning microservices

Ten services provision a joiner. A service runs only when the services it depends on have finished; a failed, skipped or revoked dependency skips what depends on it, and a service that is waiting on the candidate (for example for their address) pauses rather than fails.

An arrow means “must finish first”. A failed or revoked dependency skips what depends on it; a paused one (waiting on the candidate) pauses it.

Durations are simulated (1–13 s); the browser engine replays the real server result at 4 ticks per second, four people in parallel.



Every service: what it does, undoes and updates

Service	Provision (in order)	Deprovision (reverse)	Update (details changed)
`identity-svc` **Email & SSO** 5–9 s	1. Create Google Workspace mailbox 2. Create Okta user 3. Enforce MFA enrolment	Update directory attributes Re-render the offer with the new terms Replace the e-sign envelope	Update directory attributes
`letter-svc` **Offer letter & e-sign** 3–6 s	1. Render offer PDF 2. Send e-sign link	Void the e-sign envelope Withdraw the offer letter	(per application instead)
`access-svc` **App access & credentials** 7–13 s	1. Invite to Slack workspace 2. Add to GitHub org and teams 3. Assign Okta app groups	Remove from Slack workspace Remove from GitHub org and teams Unassign Okta app groups	(per application instead)
`bgv-svc` **Background verification** 3–6 s	1. Create vendor case 2. Submit candidate documents	Cancel the vendor case	Update the vendor case
`payroll-svc` **Payroll & statutory** 4–8 s	1. Create payroll record 2. Register PF / UAN 3. Enrol in insurance	Cancel the payroll record Withdraw the PF / UAN registration Cancel the insurance enrolment	Update the payroll record
`idcard-svc` **ID card** 2–4 s	1. Compose card artwork 2. Queue print job	Cancel the print job Deactivate the card	Reissue the card artwork
`facility-svc` **Premises & desk** 2–3 s	1. Reserve desk 2. Schedule access pass	Release the desk Revoke the access pass	Move the desk and access pass
`agenda-svc` **Day-1 agenda & buddy** 2–4 s	1. Build day-1 agenda 2. Assign buddy	Cancel the day-1 events Unassign the buddy	Reschedule the day-1 events

`collect-svc` **Document collection** 1–2 s	1. Build document checklist 2. Send magic link	Expire the magic link	Refresh the document checklist
`reminder-svc` **Reminders & escalations** 2–3 s	1. Schedule nudges 2. Arm escalation ladder	Cancel scheduled nudges and escalations	Reschedule nudges

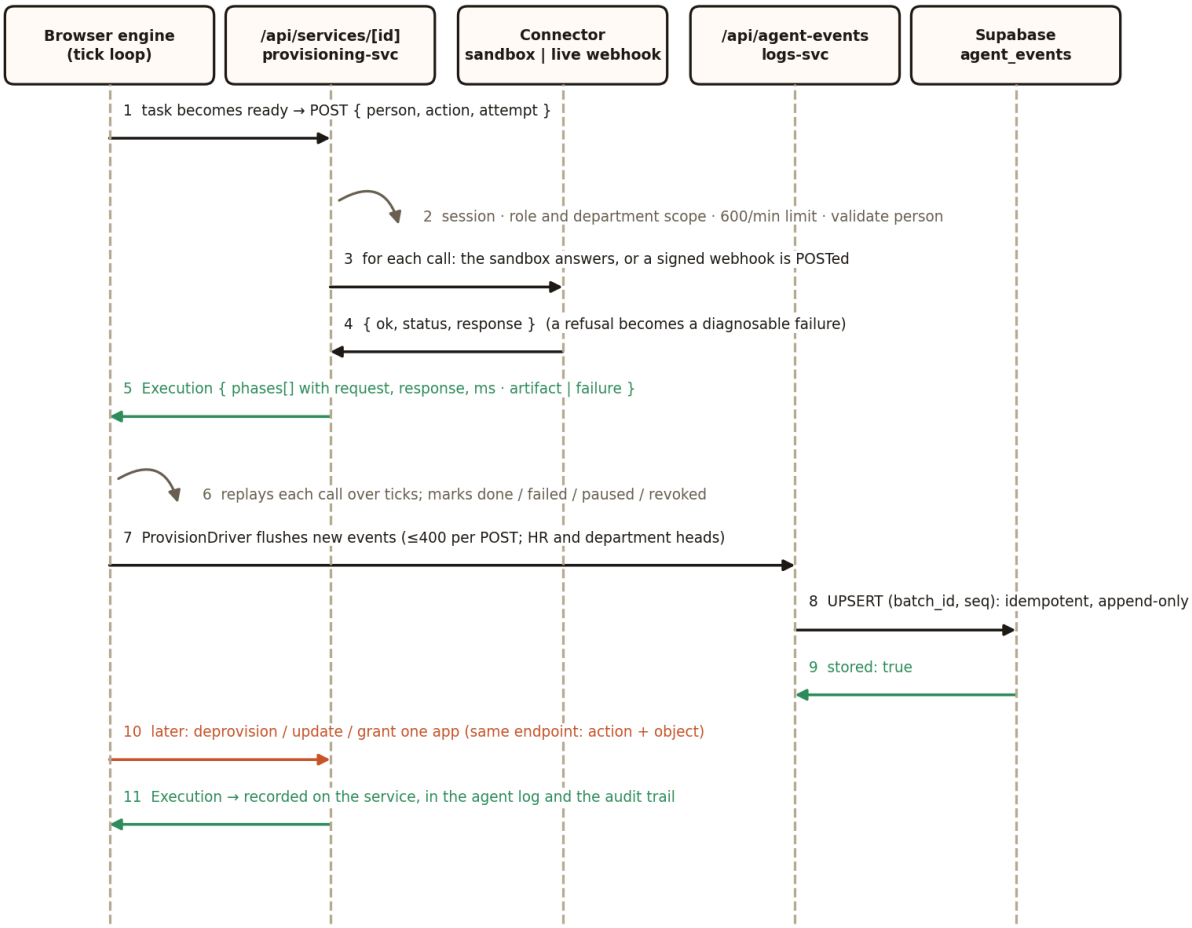
Actions and connectors

- **Actions**: `provision` (the original run, or grant one application), `deprovision` (undo it) and `update` (move or reissue what was created). Access also takes an `object`, one application name, so a single app can be revoked or granted on its own.
- **Sandbox connector** (default): answers like a healthy system, with a stable ~9% of provisioning runs failing deterministically so there is something to diagnose; a retry after the cause is fixed succeeds. Deprovision and update always succeed in the sandbox.
- **Live connector**: setting `SERVICE_WEBHOOK_<SERVICE_ID>` (for example `SERVICE_WEBHOOK_IDENTITY_SVC`) makes the service POST each call to your system. The body carries the service, trace id, attempt, action, object, the phase (index, label, call), the request body and the person; it is signed with HMAC-SHA256 in `x-superjoin-signature` using `SERVICE_WEBHOOK_SECRET`. The system answers `{{ ok, status?, response?, artifact?, paused?, error? {{ code, title, cause, fix, retryable }} }}`. Timeouts (8 s) and unreachable connectors become diagnosable failures.
- **Every execution** returns each call's label, endpoint, request body, response body, HTTP status and milliseconds, plus the artifact created or the failure (code, cause, evidence, fix, retryable) and a trace id.

Provisioning run and logging

Provisioning run and logging

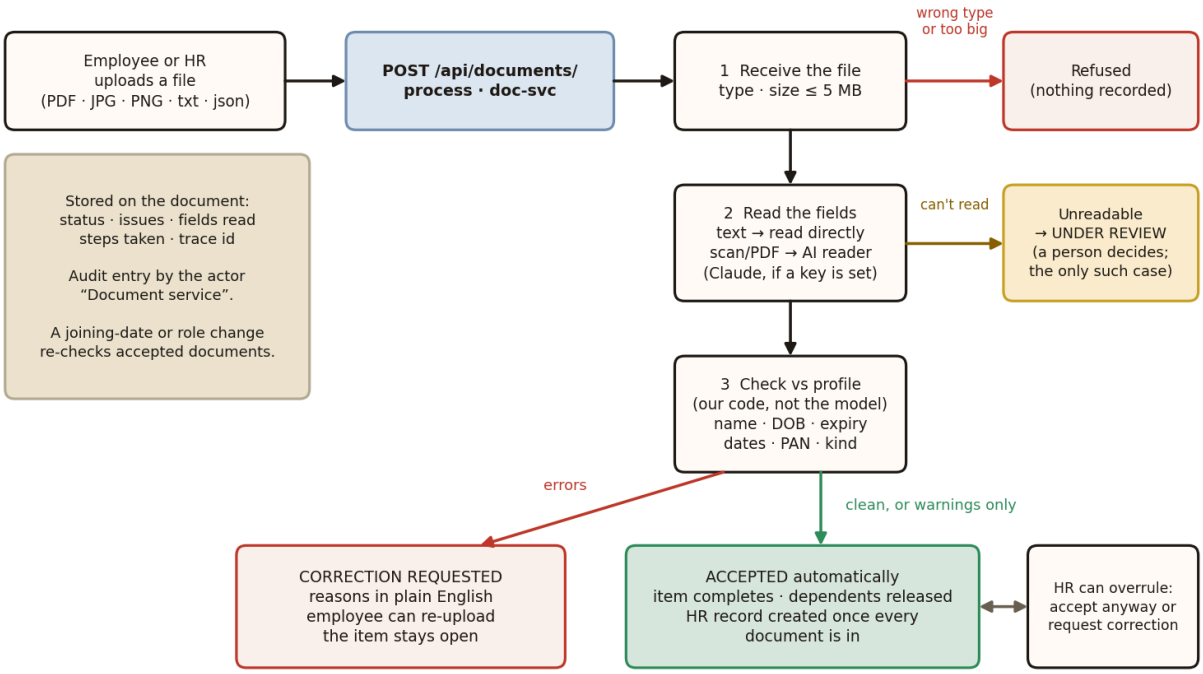
When a service becomes ready the browser engine asks the server to run it, then replays the server's result over ticks so progress looks and behaves like a real run. If the server can't be reached, the run falls back to a local simulation and the log says so.



Documents, changes and termination

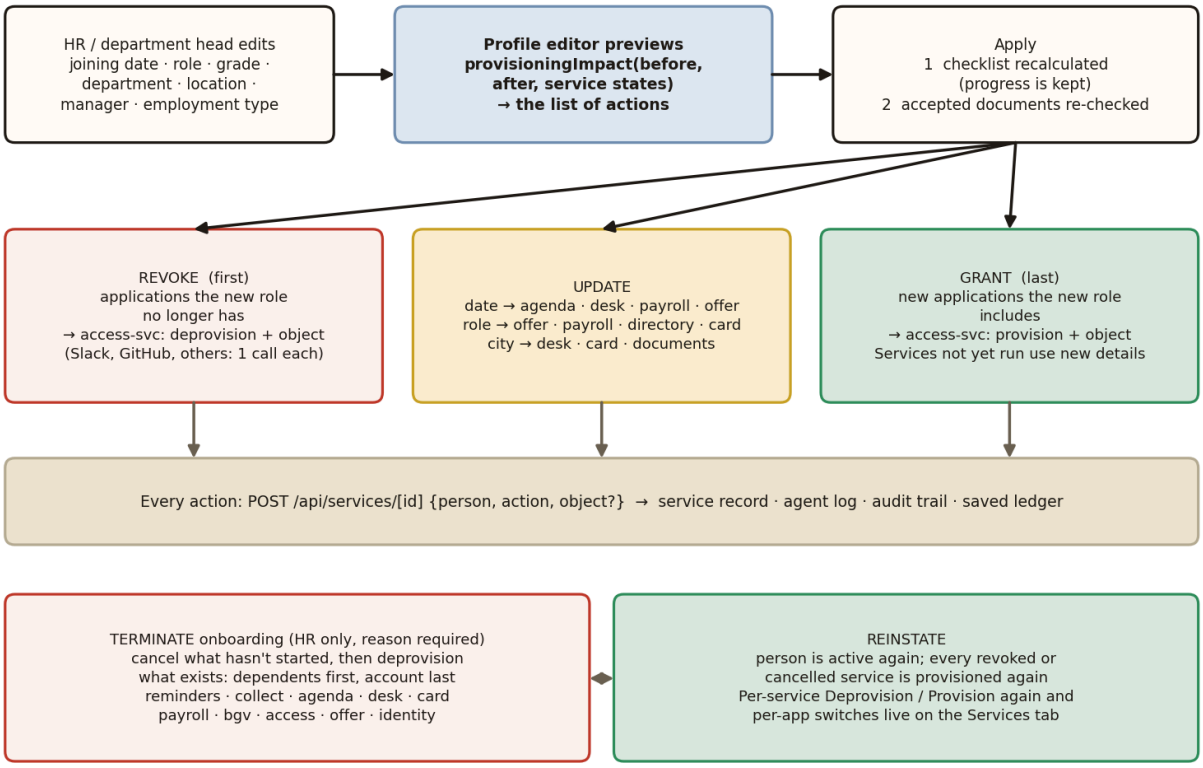
Document processing

Every upload (an employee's own, or HR on their behalf) goes through the document service, which decides. The model, when used, only reads what is printed; our own code checks it against the profile and makes every judgement.



Fallback: if the service can't be reached, the browser runs the same checks locally instead of leaving the upload hanging.

Changing a person, undoing provisioning, terminating



ATS sync, chat agent, assistant, escalation

ATS sync

- HR picks Greenhouse, Lever or Workday and presses Connect. The browser POSTs `/api/ats/[provider]` once per page (10 per page, up to 30 pages), logging each request line, status, record count and time.
- Each vendor payload is in the vendor's own format (Greenhouse applications with candidate, jobs and offer; Lever opportunities with categories and fields; Workday report entries). A provider-specific mapper turns it into the same columns a CSV uses, losslessly (tested round-trip).
- The rows go through the same validation as an upload. In ATS and form mode, people whose email is already onboarding are skipped, new joiners are added to the existing batch, checklists are built for them and provisioning starts for them only.
- Demo tenant: 19 people (the 14 already loaded plus 5 new hires). A key shorter than 8 characters, or the word invalid, gets a vendor-style 401. With `ATS_<VENDOR>_URL` and `_TOKEN` set, the server reads a live tenant with its own token and ignores the typed credentials.

Chat agent (Ask superjoin)

- The route is stateless and takes one model step: the browser sends the conversation so far (including tool results); the model answers or asks for tools; the browser runs the tools against *this user's* scope of the data (it lives in the browser) and calls again, up to six rounds.
- The system prompt and the tool list are built on the server from the verified session; the request cannot widen either. Tool-use blocks naming a tool the role doesn't have are rejected.
- Tools: `list_people`, `get_checklist`, `search_policies` (everyone); `list_escalations`, `service_status` (staff); `retry_service`, `send_escalation` (HR and department heads). Actions are shown first and run only after a Confirm click.
- Without a key, or if the model is unreachable, an offline router answers from the same tools (and says so). Policy questions are answered only when the policies actually mention the question's terms; otherwise it declines rather than guesses.

Policy assistant (per person)

- Retrieval: BM25 (k1 1.4, b 0.75) over the policy passages that apply to that person's country, department, location and level; headings and titles count extra; a small synonym table helps paraphrases.
- Coverage gate: the best passage must score at least 3.2 and cover at least 60% of the question's content words, otherwise the answer is a fixed “not covered, ask HR”. With fewer than 40 applicable passages, all of them go to the model so paraphrases still land.
- The model returns JSON (answer, covered, citations). Every citation must be a real source id; an unverifiable reply is replaced by an extractive answer that quotes the policy. The assistant also answers from the person's own checklist (status, owner, why).

Escalation and reminders

- An item is escalated when it is itself overdue and is either an approval or is holding other items up (the root cause, not the symptoms). One day overdue goes to the first rung, three days to the second; the target comes from the owner directory.
- Reminder emails follow a four-step ladder at 0, 1, 3 and 5 days: the owner; owner and manager; department head; HR head. Terminated people are never chased.

API reference: auth, health, logs, email

Eleven route files expose 16 method handlers. Every route (except login, logout and health) returns 401 with no body when there is no valid session, and every role or department check is made on the server from the verified session, never from the request. Error bodies are JSON `{{ error }}` unless noted.

POST /api/auth/login

Microservice	auth-svc
Access	Public
Request	`{ "userId": "u-priya" }`
Response	`200 { "ok": true, "landing": "/tracker" | "/me" }` Sets HttpOnly cookie sj_session (HS256, 8 h, carries only the user id)
Errors	401 { error: "Unknown user" }

POST /api/auth/logout

Microservice	auth-svc
Access	Anyone
Request	none
Response	`200 { "ok": true }` Clears the session cookie
Errors	none

GET /api/health

Microservice	health-svc
Access	Public (more for HR)
Request	none
Response	`Anyone: { "ok": true }` `HR: { ok, env, checks: { sessionSecret, supabase, ai, email }, connectors: { "<service-id>": "sandbox" | "live" } }`
Errors	none

GET /api/agent-events

Microservice	logs-svc
Access	HR; department head (rows filtered to their department on the server)
Request	`batch (latest | id) · limit (1–300, default 100) · before (seq) · kind (info,ok,warn,error) · service · person · q (≤80 chars)`
Response	`{ configured, setupRequired, batch, batches[≤20], events[], total }` events: seq, tick, at, personId, personName, dept, service, kind, text, call?, traceId?, ms?, meta?
Errors	401, 403, 502 (read failed)
Limits	Query values are sanitised before use in a filter

POST /api/agent-events

Microservice	logs-svc
Access	HR; department head (own department's events only)
Request	`{ batch: { id, startedBy, label, people }, events: IngestEvent[1–500] }` every field length-limited
Response	`200 { stored: true, … }` Idempotent: the same (batch, seq) is ignored
Errors	400 invalid; 403; 503 { stored: false, reason: not_configured | setup_required }; 502 { reason: batch_failed | insert_failed }

GET `/api/email/send`

Microservice	mail-svc
Access	Any signed-in user
Request	none
Response	`{ configured, redirected }`
Errors	401

POST `/api/email/send`

Microservice	mail-svc
Access	HR, department head, manager (not new joiners)
Request	`{ to, subject (≤200), html (≤100 KB), text (≤20 KB) }`
Response	`{ mode: "sent", id, redirectedTo }` | `{ mode: "preview", note }` | `{ mode: "failed", error }`
Errors	400, 401, 403, 413, 502

API reference: assistant, documents, ATS

GET /api/assistant

Microservice	assistant-svc
Access	Any signed-in user
Request	none
Response	`{ ai: boolean, model: string | null }`
Errors	401

POST /api/assistant

Microservice	assistant-svc
Access	Any signed-in user
Request	`{ question (≤500), employee { name, role, department, location, country, grade, level, startDate, manager }, hr { name, email }, facts[≤80], policies?[≤60, ≤150 KB], history?[≤6] }`
Response	`{ answer, covered, citations[], mode: "ai" | "extractive", note? }`
Errors	400, 401, 413, 429
Limits	20 requests/min/user · 60 s

POST /api/documents/extract

Microservice	doc-reader-svc
Access	Any signed-in user
Request	`{ mime: "application/pdf" | "image/png" | "image/jpeg", data: base64 (≤5 MB) }`
Response	`{ extracted: { docKind, name, dob, idNumber, address, issueDate, expiryDate, degree, institution, year, employer, lastWorkingDate } }` Fields the model could not read are null; it never infers
Errors	400, 401, 413, 429, 502 { extracted: null, reason: extraction_failed }, 503 { reason: not_configured } (no API key)
Limits	15/min/user · 60 s

POST /api/documents/process

Microservice	doc-svc
Access	Any signed-in user; a new joiner only for themselves
Request	`{ fileName, size, docType, person, text | { mime, data } }` docType: id_proof, address_proof, education_cert, prev_employment, pan_card, us_work_auth
Response	`{ traceId, mode, extraction, extracted?, issues[], status, steps[{label, ok, ms, detail}], rejected? }` status: Accepted | Correction Requested | Under Review (unreadable scan only) | Not Submitted (rejected file)
Errors	400, 401, 403 (not their own), 413, 429
Limits	40/min/user · 60 s

POST /api/ats/[provider]

Microservice	ats-svc
Access	HR only
Request	`{ tenant, token, page (1–30) }` provider: greenhouse | lever | workday
Response	`{ ok, mode: sandbox | live, request, status, ms, payload, records, hasNext, total? }` On a refused key: { ok: false, status: 401, error }
Errors	400, 401, 403, 404 (unknown provider), 429

Limits	120/min/user · 60 s

GET `/api/services/[service]`

Microservice	provisioning svc
Access	HR; department head
Request	none
Response	`{ id, name, creates, deps, mode, calls[{label, call}] }`
Errors	401, 403, 404

API reference: services, chat, pages

POST `/api/services/[service]`

Microservice	provisioning svc
Access	HR (any person); department head (own department only)
Request	`{ person, attempt? (1–20), retry?, action?: "provision" | "deprovision" | "update", object? }` service: identity-svc · letter-svc · access-svc · bgv-svc · payroll-svc · idcard-svc · facility-svc · agenda-svc · collect-svc · reminder-svc; object only for access-svc
Response	`{ service, traceId, mode, action, object?, attempt, ok, phases[{label, call, ok, httpStatus, ms, request, response}], paused?, artifact?, failure? { code, title, cause, evidence, fix, retryable, phase }, transient, totalMs }`
Errors	400 (bad person, action or object), 401, 403, 404, 429
Limits	600/min/user · 60 s

GET `/api/agent/chat`

Microservice	chat-agent-svc
Access	Any signed-in user
Request	none
Response	`{ ai, model, tools: string[] }` (tools offered to this role)
Errors	401

POST `/api/agent/chat`

Microservice	chat-agent-svc
Access	Any signed-in user (tools by role)
Request	`{ messages: Message[1–40] }` ≤250 KB; text ≤4000 chars; tool_use only from assistant and only for tools the role has; tool_result only from user; starts and ends with the user
Response	`{ mode: "ai", content: [text | tool_use], stop_reason }`
Errors	400, 401, 413, 429, 503 { mode: "offline" } (no key), 502 { mode: "offline", reason: model_unavailable }
Limits	30/min/user · 60 s

Pages

Path	Who can open it	Purpose
/login	anyone	Persona sign-in
/	signed-in	Redirects to the landing page for the role
/me	new joiner	Their own overview, checklist, documents and assistant
/tracker	HR, department head, manager	Progress dashboard (table, stack, grid, matrix)
/employees/[id]	HR, department head, manager (in scope)	Full record: overview, checklist, documents, profile and changes, services, escalations, activity, assistant
/builder	HR, department head, manager	Workflow builder (view only unless you may edit that department)
/library	HR, department head, manager	Workflow library by type, department and level
/ingest	HR	ATS sync, add a joiner, upload a file, reference data editors
/reminders	HR, department head, manager	Scheduled and queued reminder emails
/agents	HR, department head	Agent overview, logs per service, failures
/forbidden	anyone	Shown when a role can't open an area
/samples/*	signed-in	Sample CSV, Excel, JSON, policies and owners files (redirects to login when signed out)

Data model and storage

Core entities (TypeScript)

Entity	Key fields
Person	id, no, name, email, phone, role, persona { type, dept, level }, manager, location, grade, dob?, address?, startDate, fields (data completeness per field), terminated? { at, by, reason }
Requirement	id, item, kind (document, activity, record, provision, policy, approval), owner function, dueOffset (days from joining), when { departments, locations, countries, roles, grades, types }, dependsOn, policy / policyByCountry, docType, because
ChecklistItem	id, personId, ruleId, item, kind, owner (named contact), dueDate, dependsOn, policy, docType, status (pending, requested, completed), ticket?, reason (the rule and the facts that matched)
DocumentRecord	status (Not Submitted, Submitted, Under Review, Accepted, Correction Requested), fileName, extraction (text, ai, none), extracted, issues[], processing { traceId, mode, steps[] }, attempts, reviewer note
Policy	id, title, version, appliesTo { countries, departments, locations, grades }, sections[{ id, heading, text }]
Directory	owners[{ fn, department, location, name, email }], managers[{ name, email, department }], departments[]
Run / Task	per person: tasks[10] each with status (queued, running, retry, done, waiting, failed, skipped, revoked), actions[], execution, failure, revocation, revokedObjects, lastUpdate, revokeFailure
Execution	the server's account of one service action: phases[], artifact | failure, trace id, mode, action, object
LedgerEntry	what survives a reload: revoked services (who, why, execution) and individually revoked applications
AuditEntry	at, actor, personId, action, detail (every import, upload, decision, change, termination)

Browser storage

Key	Contents
superjoin.people	people, which reminders were sent, whether the sample batch is loaded
superjoin.onboarding	requirements, policies, directory, dataset sources, checklists, documents, audit trail, ticket counter
superjoin.provisioning	the revocation ledger only

Supabase schema (agent logs)

Table	Columns
agent_batches	id text pk, started_at timestamptz, started_by, label, people int
agent_events	id bigint identity pk, batch_id fk, seq int, tick int, occurred_at, person_id, person_name, dept, manager, service, kind (info, ok, warn, error), message, call, trace_id, ms, meta jsonb, unique (batch_id, seq)
indexes	(batch_id, seq desc), dept, service, kind, trace_id

- **Row-level security** is enabled on both tables. Every policy also requires `app_gate()`, which checks that the request carries the `x-superjoin-key` header equal to a secret known only to our server. The publishable key shipped to the browser therefore cannot read or write anything on its own (checked by `npm run db:check`).
- **Append-only**: there is no update or delete policy, and update, delete and truncate are revoked. Inserts are idempotent on (batch_id, seq).
- The migration is `supabase/migrations/20261031000000_agent_events.sql`; `npm run db:sql` prints it with your app key filled in; `npm run db:seed` stores the sample batch's history (714 events).

Security and safety

Concern	How it is handled
Authentication	HS256 session cookie (jose), HttpOnly, 8 hours, carrying only the user id. Role and scope are looked up server-side on every request. SESSION_SECRET is mandatory in production (the app refuses to sign sessions without it). The login is a demo identity provider: replace it with SSO.
Authorisation	One policy table (areas by role, edit and view rules, data scope) used by the proxy, every page, every route and the UI. Department heads are limited to their department on the server for logs and service actions; new joiners can only process their own documents.
Data scoping	Applied to people, workflows, logs, services and chat tools. People data currently lives in the browser, so scoping there is a display rule, not a boundary (see limitations).
Input	Every route validates shape and length (names, strings, file sizes, array sizes); base64 is checked; the person object sent to service routes is rebuilt from validated fields only; PostgREST filter input is sanitised.
Abuse	Per-user sliding-window rate limits on paid or heavy routes (assistant 20/min, extract 15/min, documents 40/min, chat 30/min, ATS 120/min, services 600/min).
Webhooks	Signed with HMAC-SHA256 (x-superjoin-signature) and a trace header; 8 second timeout; failures are reported, never hidden. Webhook URLs and tokens are server environment variables and are never sent to the browser.
AI safety	The model never decides: it reads printed fields or writes answers from supplied sources. Our code validates documents, verifies every citation, and substitutes a quoted answer when a reply can't be verified. Tool results and document text are treated as data, not instructions. Chat actions need a human click. Policy answers refuse when the policies don't mention the question's terms.
Secrets	Never committed (.env* is ignored). Only NEXT_PUBLIC_* values reach the browser (the Supabase URL and publishable key, which are useless without the server's gate key).
Auditability	Every import, upload, decision, change, termination and revocation is written to the audit trail with the actor; every service call is in the agent log with a trace id.

Domain logic in detail

Checklist and status

- **Rules**: 29 default requirements (documents, activities, records, provisioning requests, policy acknowledgements, approvals), each with a due date relative to joining, conditions, dependencies and a plain-English reason. Policies can differ by country (India and United States).
- **States**: completed; blocked (a dependency isn't complete, or a document was sent back); overdue (past due and not blocked); otherwise pending. **Ready for day one** means every item due on or before the joining date is complete.
- **Automation**: when the last document is accepted the agent creates the HR record and raises tickets for the laptop, email, badge and desk.

Document checks (our code)

Check	Result
File type and size	PDF, PNG, JPG, JPEG, txt or json, at most 5 MB; otherwise refused outright
Name	exact match passes; a close match (for example an added initial) is a warning; a different name is an error
Date of birth	must match the profile (error)
Document kind	a wrong kind (for example a library card as ID) is an error
Expiry (ID, work authorisation)	expires before joining: error; within six months of joining: warning
Proof of address	dated more than 3 months before joining: error; future date: warning; address not mentioning the city: warning
PAN	must be five letters, four digits, one letter
Education / employment	year of passing not after the joining year; last working day not after the joining date
Missing fields	a field the document type needs but the file lacks is an error

Changing a person: what happens to provisioning

Change	Checklist	Provisioning
Joining date	due dates move; progress kept	update: offer, background case, payroll, desk and pass, day-one calendar and buddy, nudges
Role, grade, level or type	items appear or go; owners move	revoke apps the new persona no longer has, grant new ones; update offer, payroll, card, directory, document list
Department	owners and policies change	revoke and grant apps; update directory, payroll, desk, day-one plan, nudges
Location	country-specific items change	update desk and pass, card, payroll (statutory set-up), document list, directory
Manager	manager-owned items move	update directory, day-one events, nudges

Only services that have already created something are touched; a service that hasn't run yet simply uses the new details. Revokes run first, then updates, then grants. Accepted documents are re-checked against the new facts, and one that no longer passes goes back to the employee and reopens its item.

Workflow builder and library

- Personas are type (full-time, part-time, intern) × department (Engineering, Finance, Product, Business, People) × level (Intern; Associate, Mid, Senior, Leader), each with a generated step timeline across 123 blocks (78 with real brand logos) and per-step permission levels and verifiers.
- HR edits and publishes everything. A department head edits only their own department; HR publishes. Managers view only. Each role sees only the departments the policy allows.

Roles, permissions and frontend

Permissions

Capability	Employee	Manager	Dept head	HR
See own record	yes	n/a	n/a	n/a
See people	self	direct reports	own department	everyone
Dashboard, record pages	no	yes	yes	yes
See workflows (library, builder)	no	their reports' departments	own department	all
Edit workflows	no	no	own department	all
Publish workflows	no	no	no	yes
Ingest (ATS, form, file) and reference data	no	no	no	yes
Edit an employee's details	no	no	own department	yes
Send nudges and reminders	no	yes	yes	yes
Agent logs and failure detail	no	no logs	own department	all
Retry, deprovision, provision a service or app	no	no	own department	yes
Terminate or reinstate an onboarding	no	no	no	yes
Upload documents	own	no	on behalf	on behalf
Review documents (override the service)	no	no	own department	yes
Chat agent actions (retry, escalate)	no	no	yes	yes

Front-end map

Area	Main components
shell	SessionShell (server: reads the session), Header, UserMenu, Toaster, Hydrate (restores saved data and revocations), ProvisionDriver (ticks the engine, sends service calls, streams logs), AgentChat (Ask superjoin)
dashboard / tracker	ProgressTable (filters, summary tiles, readiness chips), TrackerApp (Dashboard, Stack, Grid, Matrix), Badge (flip ID card), Journey
employee	EmployeeRecord with Overview, ChecklistView, DocumentsPanel, ProfileEditor (impact preview), ServicesPanel (switches, terminate), EscalationsPanel, AuditLog, AssistantChat
ingest	IngestApp (three sources), AtsSync, JoinerForm, DatasetsPanel, RefEditors (rules, policies, owners), IssueList, Provision
agents	AgentsApp (overview, logs, failures), ServiceList, TaskChip, TaskDetail (calls, request and response, revocation)
builder / library	React Flow canvas, palette, inspector, timeline ruler; library grid by persona
reminders	scheduled and queued emails, the escalation ladder, outbox

Configuration and deployment

Environment variables

Variable	Needed	Purpose
SESSION_SECRET	required in production	signs session cookies (openssl rand -hex 32)
NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	for agent logs	Supabase project URL and publishable key (baked in at build time: redeploy after changing)
SUPABASE_APP_KEY	for agent logs	server-held gate secret; must match the value in the SQL (openssl rand -hex 24)
ANTHROPIC_API_KEY (or ANTHROPIC_AUTH_TOKEN)	optional	chat agent, AI policy answers, scan reading; without it everything falls back
ANTHROPIC_MODEL	optional	defaults to claude-opus-5-5
RESEND_API_KEY, EMAIL_FROM	optional	real email; otherwise preview mode
EMAIL_TEST_RECIPIENT	optional	funnel every email to one inbox while testing
SERVICE_WEBHOOK_SECRET	optional	HMAC key for signing connector webhooks
SERVICE_WEBHOOK_<ID>	optional	point one provisioning service at a real system (IDENTITY_SVC, LETTER_SVC, ACCESS_SVC, BGV_SVC, PAYROLL_SVC, IDCARD_SVC, FACILITY_SVC, AGENDA_SVC, COLLECT_SVC, REMINDER_SVC)
ATS_GREENHOUSE_URL / _TOKEN, ATS_LEVER_URL / _TOKEN, ATS_WORKDAY_URL / _TOKEN	optional	read a live ATS tenant instead of the demo tenant

Deploying to Vercel

- Import the GitHub repository (framework: Next.js; no build settings to change; Node 20.9 or later). The repository's default branch is the Production Branch.
- Add the environment variables above, then deploy. Run the Supabase SQL once (`npm run db:sql`, paste into the SQL editor), then `npm run db:check` and `npm run db:seed`.
- Open `/api/health` as HR to confirm the integrations and each service's connector mode, then run `BASE_URL=https://<app>.vercel.app npm run test:e2e`.

npm scripts

Script	Does
dev / build / start / lint	standard Next.js and ESLint
test	all 12 unit suites (481 checks), no server needed
test:e2e	120 checks as each persona against a running app (BASE_URL)
samples	regenerates the sample CSV, JSON, Excel, policies and owners files
db:sql / db:check / db:seed	print the migration with your key; verify tables and that the public key alone is locked out; store the sample history

Testing and repository layout

Tests

Suite	Checks	Covers
rules	34	checklist generation, reasons, dependencies, country policies
ingest	32	CSV, Excel, JSON parsing, validation of fields, dates, manager and department references
documents	30	document extraction and every validation check, outcomes
scenarios	40	date and role changes, recalculation, escalation
assistant	67	retrieval, coverage gate, grounded answers, citation verification
answer	10	end-to-end answer paths with a fake model
store	45	persisted stores, upload flow, agent automation, audit trail
services	35	the ten service functions, validation, live connector, engine replay, fallback
agent	50	chat tools, role scoping, confirmed actions, offline router, prompts, workflow visibility
ats	49	vendor payload round-trips, merge flow, editors, demo tenant
docservice	33	the document service: every decision path, scans, settle, re-check
deprov	56	reverse and update operations, per-app, live refusals, change impact, terminate, reinstate, reload
e2e.sh	120	every endpoint and page as each role against a running app

Repository layout

Path	Contents
src/app	pages and the 11 route files under src/app/api
src/proxy.ts	the optimistic session gate
src/components	agent, agents, auth, builder, dashboard, employee, ingest, library, me, reminders, shell, tracker, ui
src/store	Zustand stores: people, onboarding, provisioning, outbox, builder, toast
src/lib	workflow and catalog (builder), csv and people, services and provision (engine), onboarding/* (rules, status, documents, docService, assistant, retrieve, escalate, change, datasets, seed), serviceRuntime/*, provisioning/*, agent/*, ats/*, auth/*, supabase/*, ai, rateLimit
supabase/ migrations	the agent-events SQL
scripts	db helpers, sample builders, brand-icon generator; scripts/tests holds the suites and the e2e script
public/samples	sample joiners (CSV, JSON, Excel, with errors), requirements, policies, owners
docs	this document and its generator (build_architecture_pdf.py, diagrams.py)

What is simulated, limits and next steps

Simulated or untested

- **No real vendor has been called.** The ATS demo tenant and every provisioning connector run in a sandbox that imitates the vendor's response format. The live service-webhook path (SERVICE_WEBHOOK_*) is tested against a local mock server (signing, actions, refusals, timeouts). The live ATS path (ATS_*_URL and _TOKEN) is implemented but has no automated test at all. Neither has been exercised against a real Greenhouse, Lever, Workday, Okta, Google, Slack or payroll system.
- **The AI paths have not run against the real API** (no key was available): they are type-checked against the SDK and tested with a fake model. Without a key, the chat agent and assistant use their offline and extractive modes, and PDFs and scans that cannot be read go to a person (Under Review).
- **Most screens were verified through their rendered HTML and the 120 end-to-end checks** rather than a visual review in a browser.
- The provisioning *timings* are simulated; the calls, requests and responses come from the server functions.

Known limits

- **People, checklists, documents and the audit trail live in each browser** (localStorage). On Vercel there is no state shared between users or devices, and role scoping of that data is a display rule, not a security boundary.
- The provisioning run is rebuilt on each load; only revocations and terminations are saved, and people added in a session have no run after a reload.
- Rate limiting is in memory, per function instance.
- The login is a demo identity provider (five personas).

Recommended next steps

- **Server-side persistence**: Supabase tables for people, checklists, documents, audit and provisioning runs behind role-checked API routes, so state is shared and scoping is enforced on the server.
- **Real identity**: SSO (Okta or Google) in place of the demo login; map groups to the five roles.
- **Real connectors**: implement the ten SERVICE_WEBHOOK endpoints against the real systems (the contract is documented above) and a live ATS tenant; add retries and idempotency keys on the vendor side.
- **Durable job queue** for provisioning runs (so a closed tab does not stop a run), and a shared rate limiter.
- **Extend the chat agent** to terminate and per-service revoke, with the same confirmation step.